

# Viajaí Bolso

## Documentação Técnica e Funcional do Projeto

App de controle de gastos em viagens — Flutter

Projeto desenvolvido como trabalho de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas, em fase de evolução para se tornar um aplicativo funcional completo.

Documento gerado e atualizado automaticamente a cada alteração relevante no código.

# 1. Visão Geral do Projeto

O **Viajaí Bolso** é um aplicativo mobile (Flutter) para controle financeiro de viagens: o usuário cadastra viagens — cada uma com destino, período e orçamento — e registra despesas associadas a cada viagem, categorizadas (alimentação, transporte, etc.), acompanhando o quanto já foi gasto em relação ao orçamento definido.

O projeto nasceu como protótipo de conclusão do 3º período do curso de Análise e Desenvolvimento de Sistemas e agora está em evolução para se tornar um aplicativo funcional completo, a ser apresentado como projeto final do curso. Este documento acompanha essa evolução: a cada mudança relevante no código, uma nova seção é adicionada ao **Histórico de Alterações** (seção 7), mantendo sempre a descrição das seções anteriores atualizada com o estado mais recente do app.

## 1.1 Principais dependências (pubspec.yaml)

| Pacote                                 | Finalidade                                                                                               |
|----------------------------------------|----------------------------------------------------------------------------------------------------------|
| supabase_flutter                       | Cliente oficial do Supabase: autenticação, banco de dados (Postgres) e armazenamento de arquivos         |
| image_picker /<br>image_picker_for_web | Seleção de fotos (perfil, viagem e comprovante de despesa), enviadas ao Supabase Storage                 |
| intl                                   | Suporte a internacionalização/formatação (incluído, uso pontual)                                         |
| http (v4)                              | Consulta à API pública de câmbio (open.er-api.com) usada na conversão de moeda da tela inicial           |
| mobile_scanner (v7)                    | Leitura do QR Code da nota fiscal (NFC-e) direto pela câmera, localmente no aparelho, sem IA nesse passo |

## 1.2 Arquitetura em camadas

O código em `lib/` está organizado em três camadas, sem uso de gerenciador de estado externo (Provider, Riverpod, Bloc) — o controle de estado é feito manualmente com `setState`:

- **constants.dart** — configurações globais: URL/chave do projeto Supabase e paleta de cores do app.
- **models/** — classes de dados puras (Despesa, Categoria, Usuario, Viagem), cada uma com método `fromJson` para converter a resposta do banco.
- **services/** — camada de acesso ao Supabase (`AuthService`, `ViagemService`, `DespesaService`, `DashboardService`, `StorageService`) e à API de câmbio (`CurrencyService`, v4), usando `supabase_flutter` e `http`. O `ChatService` (v5) também entra aqui, mas não chama nenhuma API de IA diretamente — ele aciona uma Edge Function do Supabase, que é quem de fato fala com o provedor de IA (ver seção 3.4).
- **utils/** (v4) — funções auxiliares puras, sem estado: lista de moedas e formatação de valores (`moedas.dart`) e geração do CSV de exportação de despesas (`exportacao.dart`).
- **screens/** — telas do app, que chamam os `services` diretamente (não há camada intermediária de `controller/repository`).

**Atualização (v2):** o app não depende mais de um servidor próprio — desde a migração para o Supabase (seção 7, versão 2), o backend é hospedado, acessado sempre via HTTPS, com a segurança de cada usuário só acessar os próprios dados garantida por políticas de Row Level Security (RLS) diretamente no banco (ver `supabase/schema.sql`).

## 1.3 Acesso direto ao app (versão web)

**Atualização (v3):** a cada atualização enviada ao repositório, o GitHub Actions gera automaticamente a versão web do app e publica no GitHub Pages, gerando um link direto para acesso pelo navegador, sem necessidade de instalar nada — útil para demonstração na banca (ver seção 7, versão 3):

- <https://viajaaibolso-design.github.io/viaja-ai-bolso/>

## 2. Modelos de Dados (lib/models)

### 2.1 Despesa e Categoria — `despesa.dart`

Representa uma despesa individual vinculada a uma viagem e a uma categoria.

| Campo (Despesa)        | Tipo              | Descrição                                             |
|------------------------|-------------------|-------------------------------------------------------|
| idDespesa              | String?           | Identificador único (uuid gerado pelo Supabase)       |
| descricao              | String            | Descrição da despesa                                  |
| valor                  | double            | Valor gasto                                           |
| data / hora            | String / String?  | Data e hora do gasto                                  |
| formaPagamento         | String            | Ex.: Cartão de crédito, Débito, Dinheiro, Pix, Outros |
| idViagem / idCategoria | String            | Referências (uuid) à viagem e categoria associadas    |
| categoria / icone      | String? / String? | Nome e ícone da categoria (via join com categorias)   |
| fotoUrl (v4)           | String?           | URL do comprovante anexado (Supabase Storage), RF22   |

A classe **Categoria** representa uma categoria de despesa (id, nome, ícone), lida do banco.

### 2.2 Usuario — `usuario.dart`

| Campo              | Tipo    | Descrição                                                                                      |
|--------------------|---------|------------------------------------------------------------------------------------------------|
| idUsuario          | String  | Identificador do usuário (uuid do Supabase Auth)                                               |
| nome / email       | String  | Dados básicos de cadastro                                                                      |
| fotoUrl            | String? | URL pública da foto de perfil no Supabase Storage                                              |
| moedaPadrao        | String  | Moeda padrão do usuário (default: 'BRL') — RF29, também usada na conversão da home (RF52/RF53) |
| viagemAtivaId (v4) | String? | Viagem escolhida manualmente como ativa no dashboard (RF14); null = automático                 |

### 2.3 Viagem — `viagem.dart`

| Campo                        | Tipo    | Descrição                                                                    |
|------------------------------|---------|------------------------------------------------------------------------------|
| idViagem                     | String? | Identificador da viagem (uuid)                                               |
| nome / destino               | String  | Nome e destino da viagem                                                     |
| fotoUrl                      | String? | URL pública da foto da viagem no Supabase Storage                            |
| dataInicio / dataFim         | String  | Período da viagem (AAAA-MM-DD)                                               |
| orcamento                    | double  | Orçamento definido para a viagem                                             |
| totalGasto / percentualGasto | double  | Valores agregados, calculados pela view <code>viagens_resumo</code> no banco |
| idUsuario                    | String? | Dono da viagem                                                               |

| Campo           | Tipo   | Descrição                                                                |
|-----------------|--------|--------------------------------------------------------------------------|
| moedaLocal (v4) | String | Moeda do destino (default: 'BRL') — RF52, definida ao cadastrar a viagem |

## 2.4 MensagemChat (v5) — mensagem\_chat.dart

Representa uma mensagem do assistente de IA (v5 — ver nota sobre a renumeração desse recurso na seção 6), seja do usuário ou da resposta gerada pelo Gemini. Persistida na tabela mensagens do Supabase.

| Campo      | Tipo      | Descrição                                                    |
|------------|-----------|--------------------------------------------------------------|
| conversald | String    | Conversa à qual a mensagem pertence (tabela conversas)       |
| papel      | String    | 'user' (pergunta do usuário) ou 'assistant' (resposta da IA) |
| conteudo   | String    | Texto da mensagem                                            |
| criadoEm   | DateTime? | Data/hora de envio                                           |

## 2.5 ResultadoExtracao (v6) — resultado\_extracao.dart

Representa o resultado da leitura automática de uma nota fiscal (RF48–RF51): os dados que a IA conseguiu reconhecer na imagem, prontos para pré-preencher o formulário de despesa.

| Campo           | Tipo    | Descrição                                                           |
|-----------------|---------|---------------------------------------------------------------------|
| sucesso         | bool    | Se a IA conseguiu extrair os dados com confiança suficiente (RNF21) |
| estabelecimento | String? | Nome do local reconhecido na nota, quando identificado              |
| valor           | double? | Valor total reconhecido                                             |
| data            | String? | Data da compra reconhecida (AAAA-MM-DD)                             |
| categoria       | String? | Categoria de despesa sugerida pela IA                               |
| erro            | String? | Mensagem para exibir quando sucesso é false                         |

### 3. Serviços — Integração com o Supabase (lib/services)

Desde a versão 2 (seção 7), os serviços não fazem mais requisições HTTP a um servidor próprio: eles usam o cliente `supabase_flutter` para falar direto com a autenticação, o banco Postgres e o armazenamento de arquivos do projeto Supabase. A segurança — cada usuário só acessa os próprios dados — é garantida por políticas de Row Level Security (RLS) no banco, não por checagens manuais no app (ver `supabase/schema.sql`).

#### 3.1 AuthService — autenticação e perfil

| Método                                     | Recurso do Supabase                                    | Função                                              |
|--------------------------------------------|--------------------------------------------------------|-----------------------------------------------------|
| <code>cadastrar(nome, email, senha)</code> | <code>auth.signUp</code>                               | Cria uma nova conta (dispara e-mail de confirmação) |
| <code>login(email, senha)</code>           | <code>auth.signInWithPassword</code>                   | Autentica o usuário                                 |
| <code>logoff()</code>                      | <code>auth.signOut</code>                              | Encerra a sessão (local e no Supabase)              |
| <code>recuperarSenha(email)</code>         | <code>auth.resetPasswordForEmail</code>                | Envia código de recuperação por e-mail              |
| <code>redefinirSenha(...)</code>           | <code>auth.verifyOTP + auth.updateUser</code>          | Valida o código e define a nova senha               |
| <code>getPerfil(idUsuario)</code>          | tabela <code>profiles</code>                           | Busca os dados do perfil                            |
| <code>atualizarPerfil(...)</code>          | tabela <code>profiles + auth.updateUser</code>         | Atualiza nome, e-mail, moeda e/ou foto              |
| <code>alterarSenha(...)</code>             | <code>auth.signInWithPassword + auth.updateUser</code> | Reautentica e troca a senha                         |
| <code>uploadFotoPerfil(...)</code>         | Storage (bucket fotos)                                 | Envia a foto de perfil e devolve a URL pública      |

#### 3.2 ViagemService, DespesaService e DashboardService

| Classe / Método                                             | Recurso do Supabase                              | Função                                                 |
|-------------------------------------------------------------|--------------------------------------------------|--------------------------------------------------------|
| <code>ViagemService.listar(idUsuario)</code>                | view <code>viagens_resumo</code>                 | Lista as viagens do usuário (já com totais calculados) |
| <code>ViagemService.cadastrar / atualizar / remover</code>  | tabela <code>viagens</code>                      | CRUD de viagens                                        |
| <code>ViagemService.definirViagemAtiva(...)</code> (v4)     | tabela <code>profiles</code>                     | RF14 — grava a viagem escolhida como ativa             |
| <code>DespesaService.listar(idViagem)</code>                | tabela <code>despesas (+ join categorias)</code> | Lista despesas de uma viagem                           |
| <code>DespesaService.listarCategorias()</code>              | tabela <code>categorias</code>                   | Lista categorias disponíveis                           |
| <code>DespesaService.cadastrar / atualizar / remover</code> | tabela <code>despesas</code>                     | CRUD de despesas (agora inclui <code>foto_url</code> ) |

| Classe / Método                          | Recurso do Supabase    | Função                                                                                        |
|------------------------------------------|------------------------|-----------------------------------------------------------------------------------------------|
| DashboardService.getDashboard(idUsuario) | consultas combinadas   | Monta viagem ativa (manual ou automática, v4), gastos de hoje, maior gasto e últimas despesas |
| StorageService.upload(...)               | Storage (bucket fotos) | Upload genérico (fotos de perfil/viagem, comprovantes e exportações CSV)                      |

**Observação técnica:** a checagem de dono dos dados (usuário só vê as próprias viagens/despesas) agora é feita pelo banco via RLS, não pelo código do app — isso reduz o risco de um bug no client expor dados de outro usuário. Erros ainda são tratados de forma genérica nas telas (mensagem fixa "Erro de conexão com o servidor"); mensagens mais específicas por tipo de erro continuam sendo um ponto de melhoria.

### 3.3 CurrencyService (v4) — conversão de câmbio

Novo serviço que consulta a API pública e gratuita `open.er-api.com` (sem necessidade de chave) para converter o total gasto da viagem (na moeda local) para a moeda padrão do usuário, usada na tela inicial (RF52/RF53). As cotações ficam em cache na memória do app por 6 horas, guardando também o horário da última consulta para exibir ao usuário (RNF23). Se a API estiver indisponível, a tela inicial mostra "Conversão indisponível no momento" em vez de travar ou quebrar (RNF17).

### 3.4 ChatService (v5) — assistente de IA

Serviço do assistente de viagem por IA (v5 — ver nota sobre a renumeração desse recurso na seção 6). É importante destacar uma decisão de arquitetura: **o app nunca fala diretamente com a API de IA**. A chave dessa API é um segredo de verdade — diferente da `anonKey` do Supabase, que é pública por design e só é segura porque toda consulta passa pelas políticas de RLS do banco. Se essa chave fosse colocada no código do app, ela ficaria exposta por completo na versão web (qualquer pessoa consegue abrir o DevTools do navegador e ler o código gerado). Por isso, entre o app e o provedor de IA existe uma **Edge Function do Supabase** (`supabase/functions/chat-ia`), que roda no servidor e é a única parte do sistema que conhece a chave — guardada como "secret" do projeto, nunca commitada no Git.

**Provedor de IA: Gemini (Google AI Studio).** O plano inicial era usar a API da Anthropic (Claude), mas ela exige cartão de crédito já no cadastro — mesmo para liberar o crédito gratuito de teste. Como o Google AI Studio oferece uma chave de API gratuita sem exigir cartão, o provedor foi trocado. A arquitetura (chave só no servidor, dentro da Edge Function) não muda com o provedor escolhido; se um dia for preciso trocar de novo, apenas o arquivo da Edge Function precisa mudar.

| Método / Componente                | Recurso                                  | Função                                                                                                    |
|------------------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| ChatService.obterOuCriarConversa() | tabela conversas                         | Recupera a conversa mais recente do usuário ou cria uma nova                                              |
| ChatService.listarMensagens(...)   | tabela mensagens                         | Carrega o histórico salvo ao abrir a tela                                                                 |
| ChatService.enviarMensagem(...)    | tabela mensagens + Edge Function chat-ia | Salva a pergunta, aciona a Edge Function com o histórico e o contexto da viagem ativa, e salva a resposta |

| Método / Componente          | Recurso                          | Função                                                                                                                    |
|------------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Edge Function chat-ia (Deno) | API do Gemini (Google AI Studio) | Único ponto do sistema que guarda a chave da API; monta o prompt com os dados da viagem ativa e devolve a resposta gerada |

O prompt enviado ao Gemini inclui um resumo da viagem ativa do usuário (nome, destino, orçamento, total gasto, percentual do orçamento e últimas despesas), permitindo respostas contextualizadas (ex.: "quanto já gastei hoje?"). Apenas usuários autenticados conseguem acionar a função — comportamento padrão do Supabase para Edge Functions, sem necessidade de código extra.

### 3.5 NotaFiscalService (v6/v7) — leitura de nota fiscal

Serviço de leitura automática de notas fiscais, com DOIS caminhos possíveis. O primeiro (v6) é por visão computacional: o app manda a foto (em base64) para a Edge Function `supabase/functions/extrair-nota`, que chama o Gemini em modo multimodal (lê imagem e texto juntos) para reconhecer os dados. O segundo (v7, novo) é pelo QR Code impresso na nota: a tela `EscanearQrcodeScreen` usa o pacote `mobile_scanner` para DECODIFICAR o QR localmente no aparelho (sem IA nesse passo — isso não é 'tirar uma foto do QR e perguntar pra IA', é leitura de código de barras de verdade, via `CameraX/ML Kit` no Android e `AVFoundation/Vision` no iOS); só o conteúdo já decodificado (uma URL curta) é enviado à nova Edge Function `supabase/functions/extrair-nota-qrcode`, que busca a página oficial de consulta da nota no site da Receita Estadual e pede ao Gemini (modo texto, sem visão) para ler os dados ali — uma fonte mais confiável que uma foto, por vir direto do órgão emissor. Cobertura inicial do caminho por QR Code: somente notas emitidas na Paraíba (PB), estado de testes do projeto; para qualquer outro estado, a função devolve um erro claro e a tela cai automaticamente no fluxo por foto/manual, sem travar nada. As duas Edge Functions reaproveitam o mesmo secret `GEMINI_API_KEY` já configurado para o assistente de IA — não é necessário cadastrar nenhuma chave nova.

| Método / Componente                                       | Recurso                                        | Função                                                                                                                  |
|-----------------------------------------------------------|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>NotaFiscalService.extrair(bytes)</code>             | Edge Function <code>extrair-nota</code>        | Envia a foto da nota e devolve um <code>ResultadoExtracao</code>                                                        |
| Edge Function <code>extrair-nota</code> (Deno)            | API do Gemini (multimodal)                     | Analisa a imagem e devolve estabelecimento, valor, data e categoria sugerida, em JSON                                   |
| <code>NotaFiscalService.extrairPorQrCode</code> (v7)      | Edge Function <code>extrair-nota-qrcode</code> | Envia o conteúdo do QR Code lido e devolve um <code>ResultadoExtracao</code>                                            |
| Edge Function <code>extrair-nota-qrcode</code> (v7, Deno) | Consulta SEFAZ-PB + Gemini (texto)             | Busca a nota pela chave de acesso e pede ao Gemini para ler estabelecimento, valor, data e categoria do texto da página |

Para reduzir o risco de a IA "inventar" um valor (alucinação), o prompt (nos dois caminhos) pede que o modelo classifique sua própria confiança na leitura (alta/média/baixa) e devolva campos como nulo quando não tiver certeza; a Edge Function só considera a leitura bem-sucedida quando o valor foi reconhecido e a confiança não é baixa. Quando a leitura falha ou a confiança é baixa (RNF21), a tela simplesmente cai no preenchimento manual, sem travar o cadastro da despesa — e, no caminho por foto, a imagem tirada continua sendo aproveitada como comprovante da despesa (RF22), mesmo que a extração não tenha funcionado.

## 4. Telas e Funcionalidades (lib/screens)

---

### SplashScreen

Tela inicial. Exibe a logo por 2 segundos e verifica se há uma sessão ativa no Supabase Auth. Se houver, leva direto ao mainScreen; senão, à tela de Login.

### LoginScreen

Login com e-mail e senha, opção de mostrar/ocultar senha, links para "Esqueci minha senha" e "Cadastre-se". Em caso de sucesso, o Supabase Auth já mantém a sessão salva automaticamente e o app abre o mainScreen.

### CadastroScreen

Cadastro de novo usuário (nome, e-mail, senha, confirmação de senha) com aceite obrigatório dos Termos de Uso — o texto "Termos de Uso e Política de Privacidade" agora é um link que abre a TermsScreen (v4).

### RecuperarSenhaScreen

Fluxo de recuperação de senha em duas etapas: solicitar código por e-mail e, em seguida, informar o código recebido junto com a nova senha.

### MainScreen

Shell de navegação principal, com barra inferior de 5 abas: Início (Dashboard), Viagens, Despesas, Agente (v5/v6) e Perfil.

### DashboardScreen

Tela inicial pós-login: saudação personalizada, resumo da viagem ativa — manual ou automática (RF14, v4) — na moeda local da viagem, com indicador circular de progresso (fica terracota quando o orçamento estoura), valor convertido para a moeda padrão do usuário com seletor de moeda e horário da cotação (RF52/RF53, v4), gastos do dia, maior gasto e lista das despesas mais recentes com botão "Ver todas" agora funcional (RF28, v4). Suporta pull-to-refresh.

### ViagensScreen

Lista todas as viagens cadastradas em cards (foto, nome, destino, datas, progresso do orçamento), com opções de editar e excluir cada viagem, marcar/desmarcar como viagem ativa do dashboard (ícone de estrela, RF14, v4), e botão para cadastrar uma nova.

### NovaViagemScreen

Formulário de criação/edição de viagem: foto (câmera/galeria), nome, destino, datas de início e fim, orçamento e moeda local do destino (RF52, v4).

### DespesasScreen

Seleciona uma viagem e lista suas despesas, com filtro por categoria. Ao cadastrar uma despesa nova, o usuário escolhe entre escanear uma nota fiscal por foto (a imagem é analisada por IA para pré-preencher estabelecimento, valor, data e categoria — RF48/RF49, v6 — sempre revisável antes de salvar, RF50), ler o QR Code da nota (v7, mesmo pré-preenchimento, hoje só para notas da Paraíba) ou preencher manualmente (RF51), sem exigir nota fiscal. Permite editar e excluir despesas por um formulário em painel deslizante, incluindo comprovante anexado (RF22, v4 — a própria foto da nota escaneada já vira o comprovante). Mostra o total das despesas filtradas e permite exportá-las em CSV (RF23, v4, via link do Supabase Storage).

## **PerfilScreen**

Exibe e permite editar os dados do usuário (foto, nome, e-mail), alterar a senha, escolher a moeda padrão (RF29, v4) e encerrar a sessão (logout). Dá acesso às novas telas de Configurações, Sobre o app e Suporte (RF30/RF31/RF32, v4).

## **ConfiguracoesScreen (v4)**

Preferência de notificações e atalho para os Termos de Uso (RF30).

## **SobreScreen (v4)**

Informações sobre o app (versão, descrição, contexto do TCC) e atalho para os Termos de Uso (RF31).

## **SuporteScreen (v4)**

Canal de contato (e-mail copiável) e perguntas frequentes (RF32).

## **TermosScreen (v4)**

Conteúdo real dos Termos de Uso e Política de Privacidade, atendendo à LGPD (RF07/RNF14) — antes era só um texto estático sem tela própria.

## **ChatScreen (v5/v6)**

Tela do agente de IA — renomeada nesta etapa (v6) de "Assistente de IA" para "Agente de viAgens" (título da tela, mensagem de boas-vindas e item da barra de navegação), para já alinhar o nome com a nova terminologia do levantamento de requisitos. A função por trás continua sendo a da v5 — chat simples de perguntas e respostas — não o Agente de viAgens completo (ver nota sobre a renumeração desse recurso na seção 6). Balões de mensagem (usuário à direita, agente à esquerda), indicador de "digitando..." enquanto aguarda a resposta, mensagem de boas-vindas explicando o que perguntar, e histórico persistido no banco (recarregado toda vez que a tela é aberta).

## 5. Fluxo de Navegação

---

Resumo do fluxo entre as telas do aplicativo, a partir da inicialização:

- **Splash** → verifica sessão salva → **Login** (se não houver sessão) ou **MainScreen** (se já houver uma sessão salva).
- **Login** → sucesso leva ao **MainScreen**; "Cadastre-se" leva ao **Cadastro**; "Esqueci minha senha" leva à **Recuperação de Senha**.
- **MainScreen** organiza 5 abas fixas: **Dashboard** | **Viagens** (→ Nova/Editar Viagem) | **Despesas** (formulário em painel deslizante) | **Agente** (v5, chat de IA, renomeado na v6 de "Assistente") | **Perfil**.
- **Perfil** → **Sair da conta** encerra a sessão e retorna à tela de **Login**, limpando todo o histórico de navegação.

## 6. Estado Atual e Pontos de Evolução

---

Como protótipo acadêmico, o app já demonstra o fluxo completo de cadastro, login, gestão de viagens e despesas. Os pontos abaixo estão mapeados como próximos passos para transformá-lo em um app funcional completo — úteis tanto para o planejamento das próximas entregas quanto para a banca de apresentação:

**Nota sobre a renumeração dos requisitos do Agente de IA:** o levantamento de requisitos foi atualizado entre a v5 e a v6 deste documento, e a numeração RF33–RF40 (usada nas versões 5 deste documento) mudou de significado. No levantamento atual, RF33–RF47 correspondem ao **Agente de vIAgens** — um consultor de viagens completo (entrevista prévia, roteiro com gastos estimados, indicação de passagens/hospedagem/restaurantes, apoio a trajetos de carro e orientação sobre documentação e vacinas), bem mais amplo do que o assistente implementado na v5. O ChatScreen/ChatService da v5 continuam no ar e funcionando como um assistente de perguntas e respostas sobre a viagem ativa — uma primeira versão simplificada — mas não devem ser confundidos com o Agente de vIAgens completo, cujo desenho detalhado (inclusive a decisão entre usar só a IA para estimativas ou integrar APIs reais de viagem) ainda depende de validação com o orientador, conforme o próprio levantamento exige para esse bloco de requisitos.

- **Resolvido na v2:** o backend deixou de ser um servidor fixo em IP local — agora é o Supabase, acessado via HTTPS, com autenticação e regras de acesso reais (RLS).
- **Resolvido na v3:** o app passou a ter um link direto e público, gerado automaticamente a cada atualização do código (GitHub Actions + GitHub Pages) — não é mais necessário instalar nada para demonstrá-lo. Primeiro deploy já confirmado funcionando.
- **Resolvido na v4:** nova paleta de cores, câmbio na tela inicial, seleção manual de viagem ativa, seletor de moeda padrão, telas de Configurações/Sobre/Suporte, Termos de Uso com conteúdo real, anexo de comprovante e exportação de despesas em CSV — ver seção 7, versão 4.
- **Resolvido na v5:** assistente de viagem por IA (chat simples, ver nota de renumeração acima), com a chave da API do Gemini protegida no servidor (Edge Function do Supabase) em vez de embutida no app — ver seção 7, versão 5.
- **Resolvido na v6:** leitura automática de notas fiscais por visão computacional (RF48–RF51), reaproveitando a mesma Edge Function/chave de IA do assistente — ver seção 7, versão 6.
- **Resolvido na v7:** segundo caminho de leitura de nota fiscal, agora pelo QR Code da NFC-e (decodificado localmente no aparelho, sem IA nesse passo), com uma nova Edge Function que consulta a nota direto no site da Receita e usa o Gemini só para ler o texto da página — cobertura inicial limitada a notas da Paraíba (PB), com fallback automático para o fluxo por foto/manual nos demais casos — ver seção 7, versão 7.
- **Pendente de validação com o orientador:** o Agente de vIAgens completo (RF33–RF47) — planejamento de roteiro, preços de passagem/hospedagem/restaurante, apoio a viagens de carro e documentação/vacinas — ainda depende de validação com o orientador (e, se possível, com usuários reais) antes de ser implementado, conforme o próprio levantamento de requisitos pede.
- **Pendente do lado do usuário:** tanto o assistente de IA (v5) quanto a leitura de nota fiscal (v6) só funcionam de verdade depois que a Edge Function correspondente for publicada no Supabase e o secret GEMINI\_API\_KEY estiver configurado com uma chave criada em aistudio.google.com — sem isso, as telas abrem normalmente, mas a IA não responde.
- **Parcialmente testado:** o app já foi executado de verdade pela primeira vez fora deste ambiente — cadastro de usuário funcionou e o e-mail de confirmação do Supabase chegou normalmente. Login, upload de foto, CRUD completo de viagens/despesas, as novidades da v4 (câmbio, viagem ativa, comprovante, exportação), o assistente de IA (v5) e a leitura de nota fiscal (v6) ainda não foram confirmados em uso real.
- Por padrão, o Supabase exige confirmação de e-mail para novas contas — vale revisar essa configuração no painel do projeto (Authentication) para decidir se isso é desejável na demonstração.
- Tratamento de erros de rede ainda é genérico (mensagem fixa) na maior parte das telas, sem diferenciar tipos de erro — exceção feita à conversão de câmbio, que já trata indisponibilidade da API separadamente (RNF17).

- RF23 (exportação) foi implementada em CSV, não em PDF — suficiente para abrir em Excel/Sheets, mas vale registrar essa escolha caso a banca pergunte especificamente por PDF.
- Não há persistência local (offline) dos dados de viagens/despesas — o app depende de conexão com o Supabase.
- Não há rotas nomeadas nem testes automatizados no projeto até o momento.

## 7. Histórico de Alterações

---

Esta seção registra, em ordem cronológica, as mudanças feitas no projeto a partir desta documentação — servindo como registro de evolução para a apresentação final.

### Versão 1 — Estado inicial do protótipo (baseline)

Data: 27/08/2026

Documentação inicial gerada a partir do código-fonte existente do projeto (entregue ao final do 3º período), antes do início de novas implementações. Estabelece o ponto de partida do histórico de mudanças que será usado para a apresentação final do curso.

- Levantamento completo da arquitetura atual (models, services, screens)
- Nenhuma alteração de código foi feita nesta etapa — apenas documentação do estado existente

### Versão 2 — Migração do backend para o Supabase

Data: 03/09/2026

O backend próprio do protótipo (servidor fixo em 192.168.0.9:5000) foi substituído pelo Supabase: banco de dados Postgres hospedado, autenticação (login, cadastro, recuperação de senha) e armazenamento de arquivos (fotos de perfil e de viagens), tudo acessado via HTTPS. O app Flutter agora fala direto com o Supabase pelo pacote oficial `supabase_flutter`, sem depender de nenhum servidor próprio rodando. Também foi configurado um repositório Git local no projeto para acompanhamento das mudanças pelo VSCode.

- Criado o schema do banco (`supabase/schema.sql`): tabelas `profiles`, `categorias`, `viagens` e `despesas`, com segurança por usuário via Row Level Security (RLS) e bucket de Storage para fotos
- `AuthService` reescrito para usar Supabase Auth (login, cadastro, logoff, recuperação de senha por código, alteração de senha e perfil)
- `ViagemService`, `DespesaService` e `DashboardService` reescritos para consultar o Postgres do Supabase diretamente, respeitando as políticas de RLS
- Fotos de perfil e de viagens passam a ser enviadas para o Supabase Storage (antes eram salvas como texto base64 dentro dos próprios registros)
- Identificadores (usuário, viagem, despesa, categoria) passam de números inteiros para UUID, seguindo o padrão do Supabase
- Removida a dependência de `SharedPreferences` para controlar a sessão — o Supabase Auth já mantém isso automaticamente
- Configurado repositório Git local no projeto (commits com histórico de mudanças, visível pelo VSCode)
- Ainda não testado em um app rodando de verdade (não foi possível compilar o Flutter neste ambiente) — recomenda-se rodar o app e testar cadastro/login/CRUD antes da apresentação

## Versão 3 — Publicação da versão web (link direto de acesso)

Data: 09/09/2026

Foi criado um fluxo automático (GitHub Actions) que gera a versão web do app a cada atualização enviada ao repositório e publica o resultado no GitHub Pages. Com isso, o projeto passa a ter um link direto, acessível por qualquer navegador, sem precisar instalar nada — ideal para demonstrar o app durante a apresentação do TCC.

- Criado o workflow `.github/workflows/deploy.yml`, que instala o Flutter, roda 'flutter pub get' e 'flutter build web' automaticamente a cada push na branch main
- Build da web configurado com `--base-href /viaja-ai-bolso/` para funcionar corretamente no endereço do GitHub Pages
- Publicação automática do resultado do build (pasta build/web) no GitHub Pages via actions/deploy-pages
- Necessário habilitar manualmente 'GitHub Actions' como origem em Settings > Pages do repositório (passo único, feito pela interface do GitHub)
- Link de acesso ao app: <https://viajaaibolso-design.github.io/viaja-ai-bolso/>
- Primeiro deploy confirmado com sucesso (build e publicação concluídos no GitHub Actions) — link já está no ar

## Versão 4 — Nova identidade visual, câmbio na tela inicial e RFs pendentes do novo levantamento

Data: 15/09/2026

A partir do novo levantamento de requisitos e do protótipo de telas enviados pelo usuário, esta etapa aplicou a nova paleta de cores do app e implementou os requisitos novos/pendentes que já podiam ser feitos sem depender de validação do orientador — ou seja, tudo exceto o assistente de IA (RF33–RF40) e a leitura de notas fiscais por visão computacional (RF41–RF44), que o próprio levantamento pede para validar antes de implementar.

- Nova paleta de cores "Mapa & Bússola" (índigo, verde bússola, dourado, terracota, areia), aplicada trocando só os valores em `constants.dart` — nenhuma tela precisou ser redesenhada
- RF45/RF46 — conversão de câmbio na tela inicial: cada viagem agora tem uma moeda local (definida ao cadastrar/editar a viagem), e o dashboard mostra o total gasto também convertido para a moeda padrão do usuário, com seletor de moeda e horário da última cotação (RNF23). Conversão via API gratuita `open.er-api.com`, com tratamento de erro caso o serviço esteja indisponível
- RF14 — seleção manual da viagem ativa exibida no dashboard, via ícone de estrela na lista de viagens (antes era sempre escolhida automaticamente)
- RF29 — seletor de moeda padrão funcional na tela de Perfil (também corrigido um bug em que editar o perfil ou trocar a foto resetava a moeda do usuário de volta para BRL sem querer)
- RF30/RF31/RF32 — novas telas de Configurações, Sobre o app e Suporte, antes eram botões sem função no Perfil
- RF07 — Termos de Uso e Política de Privacidade ganharam tela própria com conteúdo real (LGPD), em vez de um texto estático sem link
- RF22 — anexo de comprovante (foto) às despesas, reaproveitando o Supabase Storage já usado para fotos de perfil/viagem
- RF23 — exportação das despesas filtradas em CSV, gerado no app e disponibilizado por um link do Supabase Storage
- Banco de dados: novas colunas `viagens.moeda_local`, `despesas.foto_url` e `profiles.viagem_ativa_id` — é necessário rodar o script `supabase/migracao_v4.sql` uma vez no SQL Editor do Supabase para o projeto já existente (o app foi feito para continuar funcionando mesmo antes disso, só sem essas funcionalidades novas)
- RF33–RF40 (assistente de IA) e RF41–RF44 (leitura de nota fiscal) ficam para a próxima etapa, após validação com o orientador — quando avançar, será usada a API da Anthropic (Claude) para as duas frentes
- RF23 exporta em CSV (não em PDF) e o app ainda não foi testado rodando de verdade após esta etapa — recomenda-se testar câmbio, viagem ativa, comprovante e exportação antes da apresentação

## Versão 5 — Assistente de IA no app (RF33–RF40)

Data: 16/09/2026

Após validação do orientador, foi implementado o assistente de IA do app: uma tela de chat onde o usuário pode tirar dúvidas sobre a viagem ativa, o orçamento e os gastos, e pedir dicas de economia. O assistente usa a API do Gemini (Google AI Studio) — inicialmente o plano era usar a API da Anthropic (Claude), mas ela exige cartão de crédito já no cadastro (mesmo para o crédito gratuito de teste), então o provedor foi trocado para o Gemini, que fornece uma chave de API gratuita sem exigir cartão. O app Flutter nunca fala diretamente com a API de IA nem guarda a chave — isso passa por uma Edge Function do Supabase, que funciona como um servidor intermediário seguro, independente do provedor escolhido.

- Por que uma Edge Function? A chave da API de IA é um segredo de verdade, diferente da anonKey do Supabase (que é pública por design e protegida por Row Level Security). Se essa chave fosse colocada direto no código do app Flutter, qualquer pessoa poderia extraí-la abrindo o DevTools na versão web — por isso ela fica guardada só no servidor, como 'secret' do projeto Supabase, e nunca é commitada no Git
- Nova Edge Function supabase/functions/chat-ia/index.ts (Deno): recebe a pergunta do usuário e o histórico recente da conversa, monta um prompt de sistema com os dados da viagem ativa (nome, destino, orçamento, total gasto, últimas despesas), chama a API do Gemini (modelo gemini-3.5-flash, trocável por variável de ambiente) e devolve a resposta. Só funciona para usuários autenticados (verificação de sessão já é padrão do Supabase para Edge Functions)
- Banco de dados: novas tabelas conversas e mensagens, com Row Level Security (cada usuário só acessa as próprias conversas/mensagens) — script supabase/migracao\_v5.sql para quem já tinha o projeto criado antes desta etapa
- Novo ChatService (lib/services/chat\_service.dart): cria/recupera a conversa do usuário, salva as mensagens no banco e aciona a Edge Function
- Nova tela ChatScreen (lib/screens/chat\_screen.dart): interface de chat com balões de mensagem, indicador de 'digitando...' e mensagem de boas-vindas explicando o que perguntar
- Novo item 'Assistente' na barra de navegação inferior do app, entre Despesas e Perfil
- Leitura de nota fiscal por visão computacional (RF41–RF44) fica para a próxima etapa, seguindo a ordem combinada com o usuário
- Passo pendente do lado do usuário (fora do código): criar uma chave de API gratuita em aistudio.google.com (não exige cartão), publicar a Edge Function e configurar o secret GEMINI\_API\_KEY no painel do Supabase — sem isso, o chat fica no ar mas o assistente não consegue responder. Ainda não testado rodando de verdade

## Versão 6 — Leitura automática de nota fiscal por visão computacional (RF48-RF51) e novo levantamento de requisitos

Data: 24/09/2026

O usuário enviou um novo levantamento de requisitos e um protótipo atualizado, com uma mudança importante: o 'Agente de IA' deixou de ser um chat simples e virou o 'Agente de vIAgens', um consultor de viagens completo (entrevista prévia, roteiro com gastos estimados, indicação de passagem/hospedagem/restaurante, apoio a viagens de carro e documentação/vacinas — agora RF33 a RF47). Esse bloco inteiro continua exigindo validação com o orientador antes de ser implementado, igual da vez passada, então esta etapa focou no que já podia avançar com segurança: a leitura automática de nota fiscal (agora RF48-RF51) e a confirmação de que a conversão de câmbio já implementada atende ao que virou RF52/RF53.

- RF48/RF49 — ao cadastrar uma despesa nova, o usuário escolhe entre 'Escanear nota' ou 'Digitar manualmente'. Ao escanear (câmera ou galeria), a foto é enviada para uma nova Edge Function que usa o Gemini em modo multimodal (lê imagem e texto) para reconhecer estabelecimento, valor, data e sugerir uma categoria, pré-preenchendo o formulário
- RF50 — os dados reconhecidos aparecem nos mesmos campos editáveis do formulário, com um aviso pedindo para conferir antes de salvar — nada é salvo automaticamente sem revisão
- RF51 — o modo 'Digitar manualmente' continua disponível para quando não há nota fiscal, sem nenhuma mudança no fluxo que já existia
- RNF21 — quando a IA não consegue ler os dados com confiança (foto ruim, não é uma nota fiscal, etc.), a tela avisa e deixa os campos em branco para preenchimento manual, sem travar o cadastro da despesa
- Nova Edge Function supabase/functions/extrair-nota, que reaproveita o MESMO secret GEMINI\_API\_KEY já configurado para o assistente de IA (v5) — não é necessário cadastrar nenhuma chave nova nem pagar por um segundo serviço
- A foto escaneada da nota fiscal passa a ser usada automaticamente como o comprovante da despesa (RF22, v4) — a nota que o usuário já tirou vira a prova de pagamento, sem precisar anexar de novo
- Documentação: seções de modelos e serviços atualizadas (ResultadoExtracao, NotaFiscalService), e nota explicando a renumeração dos requisitos do Agente de IA (RF33-40 antigo → RF33-47 no levantamento atual) para não gerar confusão na leitura do histórico de versões
- RF33-47 (Agente de vIAgens completo) permanecem pendentes de validação com o orientador — o assistente de chat simples da v5 continua no ar, mas é uma versão inicial, não o Agente de vIAgens final
- Renomeada a tela e os textos do assistente de IA de 'Assistente de IA' para 'Agente de vIAgens' (título da tela, mensagem de boas-vindas e item da barra de navegação inferior, que ficou 'Agente' por limitação de espaço) — o nome já passa a refletir a terminologia do novo levantamento, mesmo a função por trás continuando a ser a versão inicial (chat simples da v5), não o Agente de vIAgens completo (RF33-47), que segue pendente de validação
- Edge Function extrair-nota publicada pelo painel do Supabase — cadastro de despesa por escaneamento de nota fiscal ainda pendente de teste com notas reais
- No teste do assistente de IA (chat), apareceu erro de conexão ('Failed to fetch') ao chamar a Edge Function chat-ia. Causa provável: o código dessa função foi reescrito para usar o Gemini (troca de provedor feita antes desta etapa), mas a versão publicada no Supabase ainda é a anterior — a função precisa ser reimplantada pelo painel (mesmo fluxo usado para a extrair-nota, colando o conteúdo atual de supabase/functions/chat-ia/index.ts) para passar a usar o Gemini de fato

## Versão 7 — Leitura de nota fiscal por QR Code (NFC-e), além da foto (RF48-RF51)

Data: 05/10/2026

O usuário perguntou se dava para ler a nota fiscal pelo QR Code em vez de só por foto, e foi claro que a ideia era decodificar o QR de verdade (como um leitor de código de barras), não tirar uma foto do QR e pedir pra IA 'enxergar' o código nela. Pesquisamos como funciona o QR Code de uma NFC-e brasileira: ele não traz o valor nem os itens da compra, só uma URL de consulta no site da Receita do estado emissor, com a chave de acesso de 44 dígitos. Por isso, a leitura virou um fluxo de duas partes — decodificação do QR localmente no aparelho (sem IA, via pacote `mobile_scanner`) e, depois, uma Edge Function no servidor que busca essa página de consulta e usa o Gemini (em modo texto, não mais visão) para ler os dados ali, o que tende a ser mais confiável que adivinhar a partir de uma foto. Também avaliamos o impacto no tamanho do app antes de implementar: a biblioteca usa APIs nativas (ML Kit/Vision) e soma só ~200 KB a 2,4 MB no Android, e nada no iOS, então não pesa no app.

- Adicionado o pacote `mobile_scanner` (^7.3.0) ao `pubspec.yaml`, para decodificar QR Code localmente, pela câmera, em Android/iOS/Web — sem precisar de rede nem de IA nesse passo
- Nova tela `EscanearQrcodeScreen` (`lib/screens/escanear_qrcode_screen.dart`): abre a câmera com uma moldura de leitura, detecta o QR Code automaticamente e devolve o conteúdo lido para a tela de despesas
- Nova Edge Function `supabase/functions/extrair-nota-qrcode`: recebe o conteúdo do QR Code, extrai a chave de acesso de 44 dígitos, identifica o estado emissor pelos 2 primeiros dígitos da chave (tabela de códigos do IBGE) e, se for uma nota da Paraíba (PB), busca a página oficial de consulta da SEFAZ-PB e pede ao Gemini (modo texto) para extrair estabelecimento, valor, data e categoria sugerida — mesmo contrato de resposta (`ResultadoExtracao`) já usado pela `extrair-nota`, reaproveitando o mesmo `secret GEMINI_API_KEY`
- Cobertura inicial limitada à Paraíba (PB), de propósito: os outros 26 estados têm páginas de consulta diferentes entre si, e implementar todas de uma vez seria desproporcional para o escopo do TCC. Para nota de outro estado (ou QR ilegível), a função devolve um erro claro e a tela de despesas cai automaticamente no fluxo por foto/manual já existente — nada trava para o usuário
- `NotaFiscalService` ganhou o método `extrairPorQrCode(qrContent)`, espelhando o mesmo padrão de segurança do `extrair()` por foto (a chave de IA nunca sai do servidor)
- `DespesasScreen`: a lógica que aplica o resultado da IA aos campos do formulário (antes só dentro da função de escanear por foto) foi extraída para `aplicarResultado()`, reutilizada tanto pelo fluxo de foto quanto pelo novo fluxo de QR Code — evita duplicar a lógica de preenchimento/aviso
- Novo botão 'Ler QR Code da nota (PB)' na tela de nova despesa, abaixo das opções de Câmera/Galeria já existentes
- `mobile_scanner` ainda não foi testado em um app rodando de verdade neste ambiente (sem Flutter instalado aqui); e a Edge Function `extrair-nota-qrcode` também precisa ser publicada pelo painel do Supabase (mesmo processo das demais) antes de funcionar de ponta a ponta